

A* Search: 15 Puzzle Solver

Programming Assignment 1

(Track B - implementation and building something)

Table of Contents

What I built.....	2
Language.....	2
The puzzle as a graph.....	2
How a board is stored.....	2
The heuristic.....	3
The search.....	3
Differences from the textbook version.....	3
Getting around the code.....	3
The tool.....	4
Interface decisions.....	4
A worked example.....	5
What I learned.....	6
Solvability test was backwards.....	6
A* is Dijkstra but with 1 extra term.....	6
2 similar boards can need very different amounts of work.....	6
Counting the blank did not produce wrong answers, but removed the guarantee.....	6
AI Use.....	7
Which tools, and how much.....	7
What I used it for.....	7
Places it was wrong.....	7
What I understood and what I took on trust.....	8
References.....	9

What I built

I implemented A* search and used it to solve the 15 puzzle, which is a 4 by 4 sliding tile puzzle. Basically, you give the program any arrangement of the tiles. It returns the shortest sequence of moves that solves the board, or reports that the board cannot be solved.

Language

The language is C++23, built with a short Makefile:

```
puzzle: main.cpp puzzle.cpp puzzle.hpp
    g++ -std=c++23 -O2 -Wall -Wextra -Wpedantic -o puzzle main.cpp puzzle.cpp
```

```
clean:
    rm -f puzzle
```

make builds the program.

make clean deletes it

-O2 matters for the timing mentioned later

The `Board` class contains this line

```
auto operator<=>(const Board&) const = default;
```

It tells the compiler to write `==`, `<` and the other comparisons for me by comparing the tile arrays.

`==` to check whether a board is the goal.

`<` because the priority queue stores pairs of (f, board) , and when 2 boards have the same f it compares the boards themselves.

The puzzle as a graph

Each complete arrangement of the tiles is one vertex.

Two vertices are joined by an edge if a single slide turns one arrangement into the other.

Every slide costs 1 move, so the shortest path through this graph is the shortest solution.

There are roughly 10^{13} arrangements, which is too many to store, so the program never builds the graph. When `solve` needs to know which boards it can reach from the board it is on, it slides the blank `'_'` up, down, left and right and calculates those boards at that point, instead of looking them up from a list built earlier.

How a board is stored

A `Board` holds a `std::array<std::uint8_t, 16>`. The 16 slots are the puzzle tiles in order, from left to right and top to bottom. The value 0 means the blank square. Each board takes 16 bytes. The search uses `manhattan`, `isSolvable`, `blank`, `slide`, and `values`. It also needs a hash function because `std::unordered_map` requires one to store boards as keys.

The heuristic

For each of the 15 tiles, `Board::manhattan` adds up how many rows and columns that tile is away from the square where it belongs.

The blank is not counted. If it were to be counted, one slide would change the total by two, because a slide moves the blank and a tile. The move itself only costs one. The estimate could then come out larger than the real number of moves left, and A* would no longer be guaranteed to return the shortest solution.

The search

`solve` is A*, the shortest-path program introduced by Hart, Nilsson and Raphael (1968), and it uses 2 structures.

Every board the search discovers carries 3 numbers (g,h,f). `g` is how many moves it took to reach that board from the starting board, which is counted. `h` is the estimate of how many moves are still needed, which is what `manhattan` returns. `f = g + h` is the estimated total length of a solution that goes through that board.

A `std::priority_queue` holds pairs of `(f, board)` for boards that have been found but not yet processed. `std::greater` makes the smallest `f` come out first.

A `std::unordered_map<Board, Node>` holds a record for every board seen. Each record stores `g`, the board it was reached from, the slide that reached it, and whether it has been processed.

`std::priority_queue` does not support decrease-key, which is the operation of lowering an entry's `f` while it sits in the queue. Instead, when a shorter route to a board is found, the board is added to the queue a second time with its new smaller `f`. That entry is processed first. When the older entry reaches the front of the queue later, the record already shows the board as processed, so it is skipped.

The search stops after three million boards, set in `maxBoardsToProcess`. This is a safeguard for boards that would otherwise use more memory than is available.

Differences from the textbook version

Every edge costs 1, so `g + w(u,v)` simplifies to `g + 1`. The textbook version uses a separate set to track which boards have been processed. Here, the `processed` flag inside each record does that instead, so one map is sufficient.

Getting around the code

`Puzzle.hpp` declares the `Board` class, its hash function, `SearchResult`, and three functions.

`Puzzle.cpp` has the `Board` methods first, then `solve`.

`Main.cpp` handles arguments, reads input and prints results.

The tool

`puzzle` is a command line solver.

It reads a board, finds the shortest solution, and prints that solution along with how many boards A* had to process to find it.

```
./puzzle solve [file]    solve a board read from a file, or from standard input
./puzzle random [seed]  scramble a board and solve it
```

`main` does no algorithm work. It reads the input into a `Board`, calls `solve`, and prints the `SearchResult`. The heuristic is a method on `Board`, so replacing Manhattan distance with a different estimate would only require changing one function.

A* is Dijkstra's algorithm with one change, a view developed by Polylog (2023). The Dijkstra from the week 6 lectures takes the unprocessed vertex with the smallest `dist_to`, which here is the number of moves made so far. A* takes the smallest $g + h$, so it also accounts for how far the board still looks from the goal. In the code that difference is one term:

```
queue.push({g + 1 + next.manhattan(), next});
//          ^^^^^  ^^^^^^^^^^^^^^^^^^^^^
//          g      h
```

If I just remove `next.manhattan()`, this becomes plain Dijkstra, which on this graph searches every direction equally and has to look at a lot more boards.

Interface decisions

2 commands.

The tool does two things: solve a given board or generate and solve a random one. Each is a separate command (`solve` and `random`) rather than a set of flags.

Input ignores whitespace

Reading with `>>`, so a board can be written on one line or as a 4x4 grid. The same code handles files and standard input.

Moves describe where the blank slides.

A move is U, D, L or R. These labels match the `slide` function directly, so no translation is needed between the search and the output.

The output includes how many boards were processed.

This shows how much work A* did to find the solution, which varies between boards.

Three separate failure messages

- Unsolvable board
- Board that hits the limit set in `maxBoardsToProcess`
- Invalid input

Each produces a different message, because each has a different cause.

A worked example

```
$ echo "1 2 3 4 5 6 7 8 9 10 0 11 13 14 15 12" | ./puzzle solve
```

```
1 2 3 4
5 6 7 8
9 10 _ 11
13 14 15 12
```

```
Solving...
```

```
Solved in 2 moves
```

```
Boards processed: 2
```

```
Time: 0.00s
```

```
Moves: R D
```

Tiles 11 and 12 are each one square from home and everything else is in place, so h is 2 for the start board.

A* processes it and works out the four boards one slide away (so moving UDLR). Each of those took 1 move, so g is 1 for all of them. Sliding the blank right (R) puts tile 11 home, which drops h to 1, giving $f = 1 + 1 = 2$. The other three pull a tile off its home square, raising h to 3, and giving $f = 1 + 3 = 4$. A* takes the smallest, so it takes the right slide. From there, sliding down reaches the goal.

The search finishes having processed 2 boards. The estimate matched the true distance here, so A* processed no unnecessary boards.

A scrambled board requires more work:

```
$ ./puzzle random 42
```

```
Scrambled with seed 42.
```

```
_ 1 4 5
3 11 2 8
6 7 9 12
13 15 10 14
```

```
Solving...
```

```
Solved in 42 moves
```

```
Boards processed: 823190
```

```
Time: 1.82s
```

```
Moves: R D L D R R R U U L D L D L U R U R D R D L L U R D L L U R D D R R U L L
D R U R D
```

42 moves, found after processing 823,190 boards.

Boards that cannot be solved are rejected before any searching happens:

```
$ echo "1 2 3 4 5 6 7 8 9 10 11 12 13 15 14 0" | ./puzzle solve
```

```
...
```

```
No solution: this board is not reachable from the goal.
```

That is the goal board with tiles 14 and 15 swapped. Without this check, A* would process boards until it hit the limit and report failure without identifying the cause.

What I learned

Solvability test was backwards

My first version of `isSolvable` returned true when inversions + `blankRowFromBottom` was even. The correct test is odd.

Every board I had tested was several moves from the goal, so the bug did not show up. On a scrambled board, I cannot tell whether 'this cannot be solved' is right or wrong without working it out by hand, and not all scrambled boards can be solved. So the wrong answer looked like a possible answer.

I found it by trying a board one slide from the goal. I could see the move myself, so when the program said the board couldn't be solved I knew that was wrong.

The goal board confirms which way it should be. The goal has zero pairs out of order and its blank on the bottom row, so its total is $0 + 1 = 1$. The goal can be reached from itself, so odd has to be the solvable case.

A* is Dijkstra but with 1 extra term

Before this project, I thought of A* as a separate algorithm. But it is kind of just Dijkstra (from week 6 teaching materials) with the estimate added. Because A* is built on Dijkstra, it keeps the same properties. One of them is that the first time a board is processed, the route to it is already the shortest possible. I did not understand this at first, as I thought skipping processed boards was just a way to save time, and was unsure whether it might cause the program to return a longer answer on some board.

2 similar boards can need very different amounts of work

Seed 123 gives a 22 move solution after processing 754 boards.

Seed 42 gives a 42 move solution after processing 823,190.

The solution is about twice as long but the work is about a thousand times greater.

I learnt that this happens because Manhattan distance assumes each tile can slide straight to its home square, passing through other tiles. But that isn't the case, usually tiles block each other. When a board has a lot of blocking, the estimate is much lower than the true number of moves. A* can't tell which direction is best, so it seems to process more boards before finding the answer.

The three million limit exists because some boards have enough blocking that the search could run out of memory without it.

Counting the blank did not produce wrong answers, but removed the guarantee

`manhattan` skips the blank. To test the effect of that line, I made a second copy that counts the blank as a sixteenth tile belonging on square 15.

Each slide moves 2 things:

- The blank
- 1 tile

If both are counted, the total can change by 2 across a move that costs 1. The estimate can then be higher than the true number of moves remaining. A* could skip a good route, and the guarantee of returning the shortest solution is lost.

I expected the program to start returning solutions that were not the shortest.

That didn't happen. I tested both versions on the same boards and they returned the same solutions. So the answers were still correct, but the mathematical guarantee wasn't there anymore. The guarantee says A* will return the shortest solution on every possible input. Without it, some board could exist where the answer is wrong. I didn't find one but testing a limited number of boards cannot prove one doesn't exist. The reason to skip the blank is that counting it lets the total change by 2 while the move costs 1, and that applies to every board.

AI Use

Which tools, and how much.

I used Claude for most of the project. It wrote the first version of the code, helped me debug, suggested test cases and explained concepts I did not understand. I would estimate it produced most of the initial code, and I spent my time testing it, fixing what was wrong, and making sure I understood it well enough to explain it.

What I used it for

- Scaffolding the implementation: the first version of the `Board` class, the A* search in `solve`, and the command line handling in `main.cpp`.
- Debugging: working through why certain boards were being rejected, which led to the solvability bug below.
- Designing test cases: it suggested edge cases I would not have thought of, such as a board one move from the goal, an already-solved board, and an unsolvable board.
- Explaining concepts: parts of the linked A* video that wasn't clear in my head, and how A* relates to the Dijkstra from week 6 + any other concepts I was stuck on whilst doing Assignment 1.
- Helping me word my thoughts for this report, which I then took parts I liked and edited it into my own words.

Places it was wrong

1. **The solvability test was backwards.** The generated `isSolvable` tested `(inversions + blankRowFromBottom) % 2 == 0`. The correct test is `== 1`. The full story for this was in the 'What I learned' section above.
2. **A missing `#include <iomanip>` in `puzzle.cpp`.** The code used `std::setw` but the include was in the wrong file. The compiler caught it immediately. It showed that

generated code can compile in one file and fail in another when functions are split differently than the AI expected.

3. The path reconstruction used a pattern I was not familiar with. The generated loop used `nodes[b].move != '\0'` to detect the start board where `'\0'` was a null character stored as a marker. The code was correct but I couldn't fully get what `'\0'` meant or why it worked. I replaced it with `b != start`, which I believe does the same thing.

What I understood and what I took on trust

I understand the search. I get why $f = g + h$ is the right thing to sort on, why an estimate never being too large means a board is finished the first time it is processed, and why skipping processed boards is safe. I found it interesting that it is related to the Dijkstra covered in week 6.

I understand the solvability test now, but I took the formula on trust at first, and that is why the backwards comparison was not caught initially. I worked out why the answer has to be odd after finding the bug.

The hash function, $h = 31 * h + \text{tile}$ over the 16 bytes, I still take on trust. I know it is a standard polynomial hash and there have been no problems in testing. But I have not thought about how often it produces collisions on the boards this search generates. If the solver were ever slower than expected, this is where I would look first.

I tested the effect of counting the blank in the heuristic. I expected it to produce longer solutions but it didn't. So I corrected the report to reflect what actually happened.

References

Hart, P.E., Nilsson, N.J. and Raphael, B. (1968) 'A formal basis for the heuristic determination of minimum cost paths', *IEEE Transactions on Systems Science and Cybernetics*, 4(2), pp. 100–107.

Lee, T. (2026) *Advanced Algorithms, Week 6: Prim's Algorithm* [Lecture slides]. Available at: <https://troyjlee.github.io/teachings/advanced-algorithms-spring-2026/>

Polylog (2023) *The hidden beauty of the A* algorithm*. Available at: <https://www.youtube.com/watch?v=A60q6dcoCjw>